Università degli Studi di Verona

Corso di Laurea in Bioinformatica

Algoritmi per Bioinformatica

e Laboratorio di Programmazione II

I seguenti appunti sono preparati tramite i lucidi del professore e il libro di riferimento

Per la parte di laboratorio il professore prende gli esercizi da:
`https://leetcode.com/problemset/algorithms/`

Anno Accademico 2025/2026

Indice

1 Fondamenti **3**

1.1 Introduzione al corso . . . 3

1.1.1 Requisiti . . . 3

1.1.2 Programma del corso . . . 3

1.1.3 Struttura del modulo . . . 4

1.1.4 Modalità di esame . . . 4

1.2 Il ruolo degli algoritmi nell'informatica . . . 4

1.2.1 Cos'è un algoritmo? . . . 4

1.3 Esempio di algoritmo ricorsivo . . . 5

1.4 Esempi di problemi computazionali da risolvere in aula . . . 10

1.5 InsertionSort e MergeSort . . . 18

1.5.1 InsertionSort . . . 18

1.6 MergeSort . . . 26

1.6.1 Metodo Divide et Impera . . . 26

1.6.2 Descrizione dell'algoritmo . . . 26

1.6.3 Pseudocodice . . . 26

1.6.4 Correttezza: Invariante di ciclo di Merge . . . 28

1.6.5 Complessità . . . 28

1.6.6 Confronto con InsertionSort . . . 29

1.7 Tasso di crescita e ordine di grandezza . . . 29

1.7.1 MergeSort (Divide et impera) . . . 30

1.7.2 Il problema dell'assunzione . . . 31

1.7.3 Variabili casuali indicatrici . . . 31

1.7.4 Algoritmi randomizzati . . . 31

1.8 Calcolo della complessità . . . 32

1.8.1 Notazione asintotica . . . 32

1.8.2 Regole per algoritmi iterativi . . . 32

1.8.3 Ricorrenze: forma generale . . . 33

1.8.4 Metodo della sostituzione . . . 33

1.8.5 Metodo dell'albero di ricorsione . . . 34

1.8.6 Master Theorem . . . 34

1.8.7 Ricorrenze non standard (telescoping) . . . 35

2 Può tornare utile **36**

2.1 Appendice A (Sommatorie) . . . 36

2.1.1 Formule fondamentali . . . 36

2.1.2 Serie geometrica . . . 37

2.1.3 Serie armonica . . . 37

2.1.4 Tecniche di manipolazione . . . 37

2.1.5 Approssimazione con integrali 38
2.2 Appendice B (Insiemi, relazioni, funzioni, grafi, alberi) 38
2.2.1 Insiemi 38
2.2.2 Relazioni 39
2.2.3 Funzioni 39
2.2.4 Grafi 40
2.2.5 Alberi 41
2.3 Appendice C (Calcolo combinatorio e probabilità) 41
2.3.1 Principi di conteggio 41
2.3.2 Permutazioni e combinazioni 42
2.3.3 Probabilità 42
2.3.4 Probabilità condizionata e indipendenza 43
2.3.5 Variabili casuali e valore atteso 43
2.4 Appendice D (Matrici) 44
2.4.1 Definizioni base 44
2.4.2 Operazioni fondamentali 44
2.4.3 Proprietà del prodotto 45
2.4.4 Determinante e matrice inversa 45
2.4.5 Applicazioni agli algoritmi 46
2.5 Domande d'esame (Appello 2) 46

1. Fondamenti

Questa sezione introduce i concetti fondamentali necessari per comprendere il resto del corso. Non preoccuparti se all'inizio alcune definizioni sembrano astratte: diventeranno chiare con gli esempi concreti nelle sezioni successive.

1.1 Introduzione al corso

1.1.1 Requisiti

- Fondamenti di matematica discreta e grafi
- Calcolo delle probabilità
- Basi di C → si studia Java (Classi e Oggetti)[1]
- Le procedure ricorsive sono come l'Ave Maria (STUDIA: c'è nell'esame)

Nota

La ricorsione è **fondamentale** in questo corso. Se non ti senti sicuro, ripassa prima di procedere: fibonacci, fattoriale, e i concetti base di chiamate ricorsive.

1.1.2 Programma del corso

- Divide et Impera
- Programmazione dinamica
- Algoritmi Greedy
- Grafi → Nodi e archi

Questi quattro argomenti sono le **tecniche algoritmiche** principali. Ogni tecnica è un "modo di pensare" per risolvere problemi complessi:

- **Divide et Impera**: spezza il problema in pezzi piccoli
- **Programmazione Dinamica**: memorizza soluzioni parziali per evitare ricalcoli
- **Greedy**: fai sempre la scelta localmente migliore
- **Grafi**: rappresenta problemi come reti di connessioni

[1] Java è il male

1.1.3 Struttura del modulo

48 ore di lezione: 4 ore a settimana + 9 ore di studio personale.[2] Non usare ChatGPT o altri strumenti simili.

I lucidi non bastano (purtroppo).

Se proprio devi, usa **Anthropic CLAUDE**.

1.1.4 Modalità di esame

- **Prima parte:** 15 domande a risposta multipla (almeno 4 su codice Java – CODE RUNNER)
- **Seconda parte:** Progettare una soluzione algoritmica
- **Se va male:** Può fare orale se ha dubbi (Ave o Maria. . .)

1.2 Il ruolo degli algoritmi nell'informatica

Prima di tuffarci nei dettagli tecnici, capiamo **perché** gli algoritmi sono importanti. Un algoritmo è semplicemente una "ricetta" per risolvere un problema al computer.

1.2.1 Cos'è un algoritmo?

Definizione 1.1 (Algoritmo)**.** Procedura di calcolo ben definita: dato un input (anche nullo), produce un output in tempo finito.

Nota

In parole semplici: un algoritmo è una sequenza di passi precisi che trasforma dei dati di partenza (input) in un risultato (output). Deve sempre finire in tempo ragionevole.
Esempio quotidiano: una ricetta di cucina è un algoritmo:

- **Input**: ingredienti (farina, uova, zucchero)
- **Passi**: mescola, cuoci a 180°C per 30 minuti
- **Output**: torta

Definizione 1.2 (Problema computazionale)**.** Relazione input/output desiderata. L'algoritmo specifica *come* ottenerla.

Differenza chiave:

- **Problema**: "Voglio ordinare una lista di numeri" (COSA)
- **Algoritmo**: "Confronta elementi a coppie e scambiali se necessario" (COME)

Il problema dice *cosa* vogliamo ottenere, l'algoritmo dice *come* farlo.

[2] Sì, lo so, è una noia, ma serve.

Definizione 1.3 (Algoritmo corretto)**.** Termina producendo il risultato corretto per ogni istanza valida.

Osservazione 1. Non sempre esiste un algoritmo: il problema della fermata è indecidibile.

Nota

Cosa significa "indecidibile"? Esistono problemi per cui è matematicamente impossibile scrivere un algoritmo generale che funzioni sempre. Il famoso "problema della fermata" chiede: "Dato un programma qualsiasi, terminerà mai?" È stato dimostrato che non esiste un algoritmo che possa rispondere a questa domanda per tutti i programmi possibili.

1.3 Esempio di algoritmo ricorsivo

Ora vediamo un esempio concreto: calcolare i numeri di Fibonacci. Questo ci permetterà di capire la differenza tra un algoritmo **corretto** (che funziona) e uno **efficiente** (che funziona velocemente).

Nota

Cosa sono i numeri di Fibonacci?
È una sequenza di numeri dove ogni numero è la somma dei due precedenti:

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \ldots$$

Definizione formale:

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2} \text{ per } n \geq 2$$

Esempio: $F_5 = F_4 + F_3 = 3 + 2 = 5$

Nota

Le convenzioni usate per il pseudocodice sono a pag. 19 del [CLRS].

Algorithm 1: FibonacciR

Input: $n \geq 0$
Output: F_n

```
if n = 0 then
    return 0;
if n = 1 then
    return 1;
return FibonacciR(n − 1) + FibonacciR(n − 2);
```

Come funziona questo algoritmo?

Traduce direttamente la definizione matematica in codice:

1. Se $n = 0$ restituisci 0 (caso base)
2. Se $n = 1$ restituisci 1 (caso base)
3. Altrimenti, calcola F_{n-1} e F_{n-2} ricorsivamente e sommali

Esempio per $n = 4$:

- Calcola $F_3 + F_2$
- $F_3 = F_2 + F_1$, quindi serve calcolare F_2 due volte!
- $F_2 = F_1 + F_0 = 1 + 0 = 1$
- Risultato finale: $F_4 = 3$

L'algoritmo è corretto? **Sì!**

Per ogni input valido, termina? **Sì, assumendo che input sia sempre un intero n $\geq$ 0**

Se sì, calcola il valore giusto? **Sì, perché l'algoritmo codifica la definizione ricorsiva.**

Quanto **tempo** richiede per terminare?

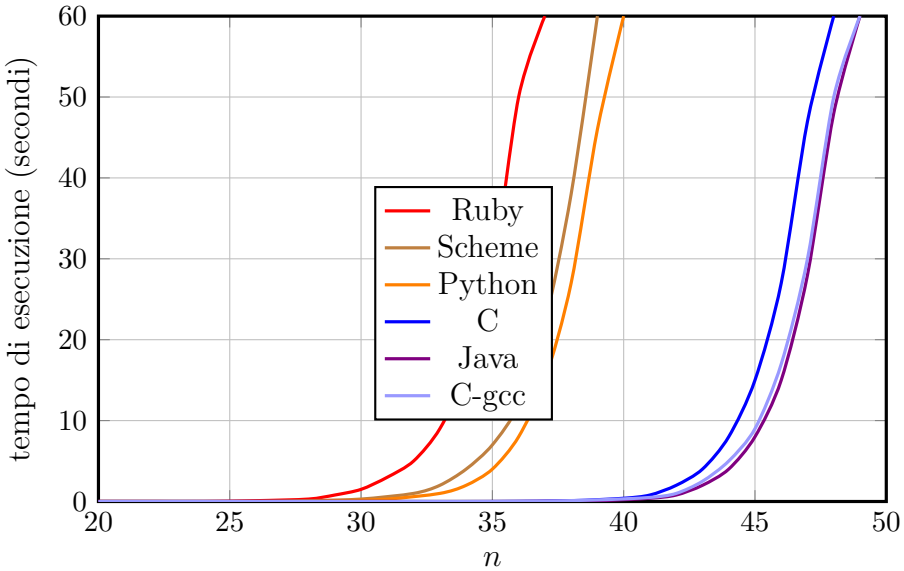


Figura 1. Confronto dei tempi di esecuzione dell'algoritmo ricorsivo `FibonacciR` in diversi linguaggi. I linguaggi interpretati (Ruby, Scheme, Python) saturano a valori di n molto più bassi rispetto ai linguaggi compilati (C, Java, C-gcc).

Problema grave: anche se l'algoritmo è corretto, è **terribilmente lento**.
Per $n = 50$ impiega diversi minuti, mentre un algoritmo migliore lo calcola istantaneamente.
Il problema è che ricalcola gli stessi valori moltissime volte.

Calcolo della complessità computazionale

La **complessità** misura quanto "costa" eseguire un algoritmo. Non misuriamo i secondi (dipendono dal computer), ma il numero di operazioni in funzione della dimensione dell'input.

Definizione 1.4 (Efficienza)**.** Risorse (tempo e spazio) richieste in funzione della dimensione n dell'input.

Definizione 1.5 (Algoritmo vs. Implementazione)**.** L'algoritmo è un concetto **astratto** ($\neq$ implementazione): descrive *come* risolvere un problema indipendentemente dal linguaggio. L'implementazione è un **programma concreto**, ovvero la traduzione dell'algoritmo in un linguaggio comprensibile dalla macchina.

Nota

Analogia:

- **Algoritmo** = ricetta scritta in italiano ("mescola gli ingredienti")
- **Implementazione** = ricetta tradotta in giapponese per un cuoco giapponese

L'idea (algoritmo) è la stessa, cambia solo la forma (implementazione).

Nozione 1.1 (Algoritmo efficiente)**.** Andiamo a cercare l'algoritmo efficiente:

- **Efficiente:** usa poche risorse
- **Risorse:** tempo e spazio richiesti dall'algoritmo per produrre l'output

Teorema 1.1 (Algoritmo vs. hardware)**.** *Con $n = 10^7$ interi da ordinare:*

$$t_A = \frac{2(10^7)^2}{10^{10}} \approx 5.5 \text{ ore} \quad (\textit{InsSort}, 10^9 \text{ istr/s})$$
$$t_B = \frac{50 \cdot 10^7 \log_2 10^7}{10^6} \approx 20 \text{ min} \quad (\textit{MergeSort}, 10^6 \text{ istr/s})$$

L'algoritmo migliore batte l'hardware più veloce.

Lezione fondamentale: un algoritmo efficiente su un computer lento batte un algoritmo inefficiente su un computer veloce.
Nell'esempio:

- Computer A: 10× più veloce, ma usa InsertionSort (algoritmo lento)
- Computer B: 10× più lento, ma usa MergeSort (algoritmo veloce)

Risultato: B finisce in 20 minuti, A impiega 5.5 ore!

Possiamo fare meglio?

Ottimizzazione: FibonacciI

Algorithm 2: FibonacciI

Input: Intero positivo n

Output: Il valore F_n

```
if n = 0 then
    return 0;
if n = 1 then
    return 1;
pprev = 0;
prev = 1;
for i = 2 to n do
    f = prev + pprev;
    pprev = prev;
    prev = f;
return f;
```

Come funziona FibonacciI (iterativo)?
Invece di chiamare ricorsivamente la funzione, mantiene in memoria solo gli ultimi due valori e li aggiorna ad ogni passo:

- Inizia con $F_0 = 0$ e $F_1 = 1$
- Calcola $F_2 = F_1 + F_0 = 1$
- Calcola $F_3 = F_2 + F_1 = 2$
- Calcola $F_4 = F_3 + F_2 = 3$
- E così via fino a F_n

Vantaggio: ogni numero viene calcolato **una sola volta**, non milioni di volte come nella versione ricorsiva!

La complessità è:

$$T(0) = 2, \quad T(1) = 3, \quad T(n) = 4 + 2n + 4(n-1) + 1 = 6n + 1$$

La crescita è **lineare in** n.

Algoritmo	Complessità	$n = 50$
Ricorsivo	$(1.4)^n$	minuti
Iterativo	$\Theta(n)$	istantaneo

Confronto drammatico:

- FibonacciR (ricorsivo): per $n = 50$ fa circa $2^{25} \approx 33$ milioni di chiamate $\rightarrow$ minuti
- FibonacciI (iterativo): per $n = 50$ fa esattamente 50 iterazioni $\rightarrow$ istantaneo

Questo dimostra che due algoritmi che risolvono lo stesso problema possono avere prestazioni **radicalmente diverse**.

Modello RAM: ogni operazione elementare ha costo 1; parola di memoria a dimensione fissa.

Nota

Il **modello RAM** (Random Access Machine) è una semplificazione che assumiamo per analizzare gli algoritmi:

- Ogni operazione base (somma, confronto, assegnazione) costa 1
- La memoria è infinita e accedervi costa 1
- Un solo processore esegue un'istruzione alla volta

Non è realistico (nella realtà ci sono cache, processori multipli, ecc.), ma è utile per confrontare algoritmi in modo uniforme.

1.4 Esempi di problemi computazionali da risolvere in aula

Ora vediamo alcuni problemi classici che ti aiuteranno a capire come si progettano e analizzano gli algoritmi. Studia bene questi esempi: sono fondamentali per l'esame!

Indici dei due numeri che sommati danno N

Problema: Data una matrice di interi `nums` e un intero `target`, restituisci gli indici dei due numeri tali che la loro somma sia uguale a `target`. Puoi assumere che ogni input abbia esattamente una soluzione, e non puoi usare lo stesso elemento due volte. Puoi restituire la risposta in qualsiasi ordine.

Nota

In parole semplici: hai un array di numeri e un numero obiettivo. Devi trovare due numeri nell'array che sommati danno il numero obiettivo, e restituire le loro posizioni.
Esempio:

- Input: `nums = [2, 7, 11, 15]`, `target = 9`
- Output: `[0, 1]`
- Spiegazione: `nums[0] + nums[1] = 2 + 7 = 9`

Algorithm 3: twoSum (Brute Force)

Input: `nums[0...n-1]`, `target`

Output: {i, j} tali che `nums[i] + nums[j] = target`, $i \neq j$

```
for i = 0 to n − 2 do
    for j = i + 1 to n − 1 do
        if nums[i] + nums[j] == target then
            return {i, j};
```

Come funziona l'algoritmo?

Prova tutte le possibili coppie di numeri:

1. Fisso il primo numero (indice i)
2. Controllo tutti i numeri successivi (indice $j > i$)
3. Se $nums[i] + nums[j] = target$, ho trovato la coppia!

Si chiama "brute force" (forza bruta) perché prova semplicemente tutte le combinazioni possibili.

Invariante: Dopo ogni iterazione esterna, tutti gli elementi da 0 a $i - 1$ sono stati confrontati con tutti gli elementi successivi.

Nota

L'**invariante di ciclo** è una proprietà che rimane vera ad ogni iterazione. Serve per dimostrare che l'algoritmo è corretto. In questo caso: "alla fine dell'iterazione i, abbiamo controllato tutte le coppie che iniziano con indici da 0 a i."

Complessità:

- Caso migliore: $\Theta(1)$ (se la coppia è trovata subito)
- Caso peggiore: $\Theta(n^2)$

Calcolo della complessità temporale (caso peggiore):

L'algoritmo esegue due cicli annidati. Il ciclo esterno itera $n - 1$ volte, e per ogni i il ciclo interno itera $n - 1 - i$ volte. Quindi:

$$T(n) = \sum_{i=0}^{n-2}(n - 1 - i) = \sum_{k=1}^{n-1} k = \frac{(n-1)n}{2} \in \Theta(n^2)$$

Perché $\Theta(n^2)$?
Il ciclo esterno fa n passi, e per ognuno il ciclo interno fa in media $n/2$ passi. In totale: $n \times n/2 = n^2/2$ operazioni. Siccome ignoriamo le costanti, rimane n^2.
Esempio concreto: con $n = 100$ elementi, l'algoritmo fa circa $100 \times 100/2 = 5000$ confronti.

La complessità spaziale è $\Theta(1)$, poiché usiamo solo variabili costanti.

Codice JAVA

```java
class twoSum {
    public int[] twoSum(int[] nums, int target) {
        for (int i = 0; i < nums.length; i++) {
            for (int j = i + 1; j < nums.length; j++) {
                if (nums[i] + nums[j] == target) {
                    return new int[]{i, j};
                }
            }
        }
        return new int[]{};  // nessuna soluzione trovata
    }
}
```

Nota

Nota sul codice Java:

- `nums.length` restituisce la lunghezza dell'array
- `new int[]{i, j}` crea un nuovo array con due elementi
- Il `return` interrompe immediatamente l'esecuzione quando trova la soluzione

Problema dell'inversione numerica

Questo problema insegna a manipolare le cifre di un numero usando aritmetica modulare. È un classico per imparare operazioni su interi.

Algorithm 4: Inversione numerica

Input: Intero x rappresentato con segno a 32 bit

Output: Intero y con le cifre di x invertite

```
y = 0;                                  // Inizializza risultato
sign = 1;                               // Memorizza segno
sign = (x < 0)? − 1 : 1;                // Memorizza segno di x
x = |x|;                                // Rende x positivo
while x > 0 do
    digit = x mod 10;                   // Estrae ultima cifra
    y = y × 10 + digit;                 // Aggiunge a y
    x = ⌊x/10⌋;                         // Rimuove cifra
return y × sign;                        // Restituisce con segno
```

Come funziona l'inversione?

Esempio: inverti 1234

1. $digit = 1234 \bmod 10 = 4$, $y = 0 \times 10 + 4 = 4$, $x = 123$
2. $digit = 123 \bmod 10 = 3$, $y = 4 \times 10 + 3 = 43$, $x = 12$
3. $digit = 12 \bmod 10 = 2$, $y = 43 \times 10 + 2 = 432$, $x = 1$
4. $digit = 1 \bmod 10 = 1$, $y = 432 \times 10 + 1 = 4321$, $x = 0$

Trucchi usati:

- $n \bmod 10$ estrae l'ultima cifra
- $\lfloor n/10 \rfloor$ rimuove l'ultima cifra
- Moltiplicare per 10 e sommare "costruisce" il numero invertito

Complessità. Sia k il numero di cifre decimali di $|x|$. Allora

$$T(k) = \begin{cases} 6 & \text{se } x = 0, \\ 5k + 6 & \text{se } x > 0, \\ 5k + 8 & \text{se } x < 0, \end{cases} \qquad \Rightarrow \qquad T(k) = \mathcal{O}(k).$$

La complessità dell'algoritmo è lineare nel numero di cifre.

Nota

Se il numero ha k cifre, il ciclo while viene eseguito esattamente k volte (una per ogni cifra). Quindi la complessità è $O(k)$, dove $k = \log_{10}(n)$ per un numero n.
Esempio: il numero 12345 ha 5 cifre, quindi $k = 5$ e il ciclo fa 5 iterazioni.

Nota

L'**operatore ternario** è definito come:
`x = (espressione booleana) ? v1 : v2`

- se l'espressione booleana è vera viene usata `v1`
- se l'espressione booleana è falsa viene usata `v2`

Esempio: `max = (a > b) ? a : b` assegna a `max` il valore più grande tra `a` e `b`.

Container con più acqua

Questo è un problema di ottimizzazione: dobbiamo trovare il rettangolo di area massima. Introduciamo la tecnica dei "due puntatori" che è molto utile per problemi su array.

Si consideri un array di interi non negativi *height* di lunghezza n. Per ogni i, la retta verticale ha estremi $(i, 0)$ e $(i, height[i])$. Si vogliono trovare due rette che, insieme all'asse x, formino un contenitore che possa contenere la massima quantità d'acqua.

Nel nostro esempio poniamo

$$A = [4, 6, 3, 5].$$

L'area del contenitore formato dalle rette in posizione i e j $(i < j)$ è data da

$$\text{area}(i, j) = (j - i)\ \min(A[i], A[j]).$$

Nota

Perché questa formula?
L'area di un rettangolo è: base $\times$ altezza

- **Base** = distanza orizzontale = $(j - i)$
- **Altezza** = il lato più basso = $\min(A[i], A[j])$

Usiamo il minimo perché l'acqua "trabocca" dal lato più basso, quindi l'altezza effettiva è limitata dalla barra più corta.

Per $A = [4, 6, 3, 5]$ otteniamo:

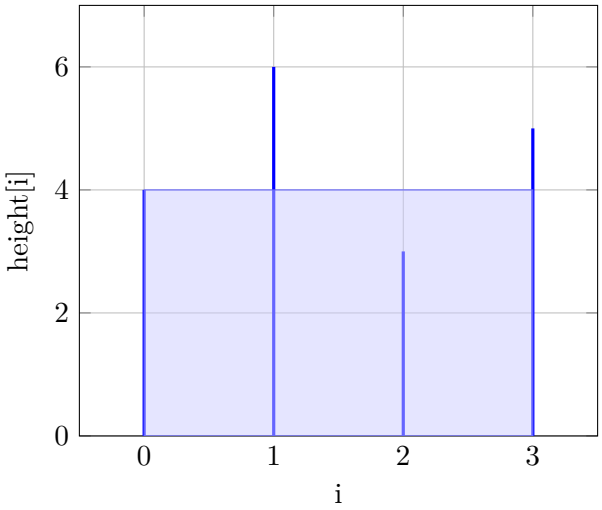


Figura 2. Container con area massima per $A = [4, 6, 3, 5]$.

Il valore massimo è quindi

$$\max_{0 \le i < j \le 3} \text{area}(i, j) = 12,$$

ottenuto per la coppia di indici $(i, j) = (0, 3)$.

Nell'esempio: $\text{area}(0, 3) = (3 - 0) \times \min(4, 5) = 3 \times 4 = 12$ è la massima possibile.

Quindi il pseudocodice per questo problema computazionale è il seguente:

Algorithm 5: Container With Most Water

Input: Array $height[0 \dots n-1]$ di interi non negativi

Output: Massima area d'acqua contenibile

```
left ← 0;                                          // Puntatore sinistro
right ← n − 1;                                     // Puntatore destro
maxArea ← 0;                                       // Area massima trovata finora
while left < right do
    width ← right − left;                          // Larghezza del contenitore
    h ← min(height[left], height[right]);          // Altezza = lato più basso
    area ← width · h;                              // Area del contenitore corrente
    maxArea ← max(maxArea, area);                  // Aggiorna massimo
    if height[left] < height[right] then
        left ← left + 1;                           // Sposta il lato più basso verso il centro
    else
        right ← right − 1;                         // Sposta il lato più basso verso il centro
return maxArea;                                    // Restituisce l'area massima
```

Tecnica dei due puntatori:

1. Partiamo dai due estremi dell'array (left=0, right=n-1)
2. Calcoliamo l'area del contenitore corrente
3. Spostiamo verso il centro il puntatore che punta alla barra **più bassa**
4. Ripetiamo fino a quando i puntatori si incontrano

Perché muoviamo il lato più basso? Se muovessimo il lato più alto, la larghezza diminuirebbe e l'altezza resterebbe limitata dal lato basso: l'area peggiorerebbe sicuramente. Muovendo il lato basso abbiamo la *possibilità* di trovare un lato più alto che migliori l'area.

Complessità. Ad ogni iterazione del ciclo `while` viene spostato solo uno dei due puntatori, che possono muoversi ciascuno al più $n-1$ volte. Pertanto il numero di iterazioni è al più $n-1$ e il costo è

$$T(n) = c_1 + c_2(n-1) = \Theta(n).$$

Lo spazio aggiuntivo utilizzato è costante, quindi $S(n) = \Theta(1)$.

Nota

Complessità $\Theta(n)$ significa che l'algoritmo è **lineare**: raddoppiando la dimensione dell'input, il tempo raddoppia. Molto meglio di $\Theta(n^2)$ dove raddoppiando l'input il tempo quadruplica!

Prefisso più lungo comune fra più sequenze

Questo problema è utile per capire come elaborare stringhe. L'idea è trovare la parte iniziale condivisa da tutte le stringhe in input.

Algorithm 6: Prefisso comune più lungo

Input: Array $S[1 \dots n]$ di stringhe

Output: Prefisso più lungo comune a tutte le stringhe

```
// Assumo che le stringhe di S siano tutte diverse da null
p ← S[1];                                   // Prefisso corrente
for i ← 2 to n do
    p ← PrefissoComune(p, S[i]);            // Prefisso tra p e S[i]
    if p = stringa vuota then
        return p;                           // Nessun prefisso comune
return p;                                   // Prefisso comune più lungo
```

Come funziona?

Esempio: $S =$ ["fiore", "fiorire", "fioraio"]

1. $p =$ "fiore" (prima stringa)
2. PrefissoComune("fiore", "fiorire") = "fiori"
3. PrefissoComune("fiori", "fioraio") = "fior"
4. Risultato: "fior"

L'idea è semplice: confrontiamo il prefisso corrente con ogni stringa successiva, accorciandolo se necessario.

Invariante: Dopo ogni iterazione del ciclo, p è il prefisso comune più lungo delle prime i stringhe.

Complessità. Sia n il numero di stringhe in S e sia m la lunghezza della stringa più corta. Ad ogni passo del ciclo la funzione `PrefissoComune` confronta al più m caratteri, quindi

$$T(n, m) = c_1 + c_2(n-1)m = \Theta(nm).$$

Lo spazio aggiuntivo utilizzato è costante, dunque $S(n, m) = \Theta(1)$.

Nota

La complessità $\Theta(nm)$ significa che nel caso peggiore confrontiamo m caratteri per ognuna delle n stringhe. Se abbiamo 100 stringhe di 50 caratteri, facciamo al massimo $100 \times 50 = 5000$ confronti.

1.5 InsertionSort e MergeSort

Ora studiamo due algoritmi di ordinamento fondamentali. InsertionSort è semplice ma lento, MergeSort è più complesso ma veloce. Capirai la differenza tra approccio **incrementale** e **divide et impera**.

1.5.1 InsertionSort

Inserisce ogni elemento nella posizione corretta tra quelli già ordinati (come le carte da gioco).

Nota

Analogia con le carte: quando ordini una mano di carte, prendi una carta alla volta e la inserisci nella posizione giusta tra quelle già ordinate. Questo è esattamente quello che fa InsertionSort!

Algorithm 7: InsertionSort

Input: $A = \langle a_1, \ldots, a_n \rangle$, array di elementi confrontabili con $\leq$
Output: A ordinato rispetto a $\leq$

```
for i = 2 to n do
    key = A[i] ;            // Inserisce A[i] nella parte ordinata a_1,...,a_{i-1}
    j = i - 1;
    while j > 0 and A[j] > key do
        A[j + 1] = A[j];
        j = j - 1;
    A[j + 1] = key;
return A
```

Come funziona InsertionSort passo passo:

1. Parte dall'elemento in posizione 2 (il primo è già "ordinato")
2. Prende l'elemento corrente (`key`)
3. Lo confronta con gli elementi precedenti (da destra a sinistra)
4. Sposta gli elementi più grandi verso destra
5. Inserisce `key` nella posizione corretta
6. Ripete per tutti gli elementi

Nota

Ordina gli elementi **sul posto** (in-place), cioè non usa array aggiuntivi ma modifica direttamente l'array di input.

Ciclo Algoritmo

5	2	4	6	1	$= A$

Ogni blocco rappresenta un ciclo for (0 o più cicli while)

1	*2*	*3*	*4*	*5*	*Commento*
5 j	2 i	4	6	1	Seq. ord.: 5 Scambio 5→2
2 ←	5 j	4 i	6	1	Seq. ord.: 2,5 Scambio 5→4
2	4 ←	5 j	6 i	1	Seq. ord.: 2,4,5 Nessun scambio
2	4	5 ←	6 j	1 i	Seq. ord.: 2,4,5,6 1 → pos. I
1	2	4	5	6 i	Seq. ordinata

Nella tabella:

- Rosso = elemento che stiamo inserendo (`key`)
- i = indice elemento corrente
- j = indice di confronto (parte da $i - 1$ e va a ritroso)
- Le frecce ← mostrano il movimento degli elementi verso destra

Invariante di ciclo

Definizione 1.6 (Invariante di ciclo)**.** Un'asserzione (proprietà booleana) sempre vera all'inizio di ogni iterazione di un ciclo.

Nota

L'invariante di ciclo è come una **promessa** che l'algoritmo mantiene: "prima di ogni giro del ciclo, questa proprietà è vera". Serve per dimostrare matematicamente che l'algoritmo funziona correttamente.

La validità si dimostra in tre passi:

1. **Inizializzazione:** è vera prima della prima iterazione.
2. **Conservazione:** se è vera prima di un'iterazione, rimane vera prima di quella successiva.
3. **Conclusione:** quando il ciclo termina, l'invariante fornisce una proprietà utile a dimostrare la correttezza.

Esempio 1.1 (Invariante di InsertionSort). *"All'inizio del ciclo* `for`*, il sottoarray* $A[1 : i-1]$ *è ordinato e formato dagli stessi elementi che erano in* $A[1 : i-1]$*."*

- **Inizializzazione:** $i = 2$, quindi $A[1 : 1]$ è un solo elemento $\Rightarrow$ ordinato. ✓
- **Conservazione:** `key`$= A[i]$, il ciclo `while` sposta $A[i-1], A[i-2], \ldots$ a destra finché non trova la posizione corretta per `key`. Quindi $A[1 : i]$ diventa ordinato con gli stessi elementi. All'iterazione successiva i aumenta di 1, l'invariante vale su $A[1 : i-1]$. ✓
- **Conclusione:** il ciclo termina con $i = n + 1$. Per l'invariante, $A[1 : n]$ è ordinato con gli stessi elementi $\Rightarrow$ l'algoritmo è corretto. ✓

In parole semplici:

- **Inizializzazione**: all'inizio funziona (un elemento è sempre ordinato)
- **Conservazione**: se funziona al passo i, funziona anche al passo $i + 1$
- **Conclusione**: alla fine, l'intero array è ordinato

È come l'effetto domino: se il primo pezzo cade e ogni pezzo fa cadere il successivo, alla fine cadono tutti!

Modello RAM

Per analizzare la complessità si usa un modello astratto di computer: la **Random Access Machine (RAM)**.

- Un solo processore con memoria ad accesso casuale.
- Numeri interi e a virgola mobile con dimensione fissa (es. 64 bit per parola).
- Istruzioni elementari (lettura, scrittura, somma, sottrazione, moltiplicazione, divisione, confronto, `if`, salto, chiamata a procedura).
- Ogni istruzione ha **costo costante** $= 1$.

Nota

È una semplificazione: nella realtà cache, pipeline e parallelismo influenzano i tempi reali. In questo corso si ignorano questi dettagli.
Perché usiamo il modello RAM? Perché ci permette di confrontare algoritmi in modo uniforme, indipendentemente dall'hardware specifico su cui girano.

Dimensione dell'input e tempo di esecuzione

Definizione 1.7 (Dimensione dell'input)**.** Misura della quantità di bit/word necessari per rappresentare l'input. Un indicatore è **corretto** se la dimensione reale è in relazione polinomiale con esso.

Definizione 1.8 (Tempo di esecuzione)**.** Numero totale di operazioni elementari eseguite dall'algoritmo nel modello RAM, in funzione della dimensione n dell'input.

Nota

Indicatore corretto: una misura semplificata della dimensione dell'input che mantiene una relazione ragionevole con la dimensione reale.
Esempi:

- Per un array di n numeri: indicatore $= n$ (numero di elementi)
- Per una matrice $n \times n$: indicatore $= n$ (anche se contiene n^2 elementi)
- Per un numero intero: indicatore = numero di bit necessari a rappresentarlo

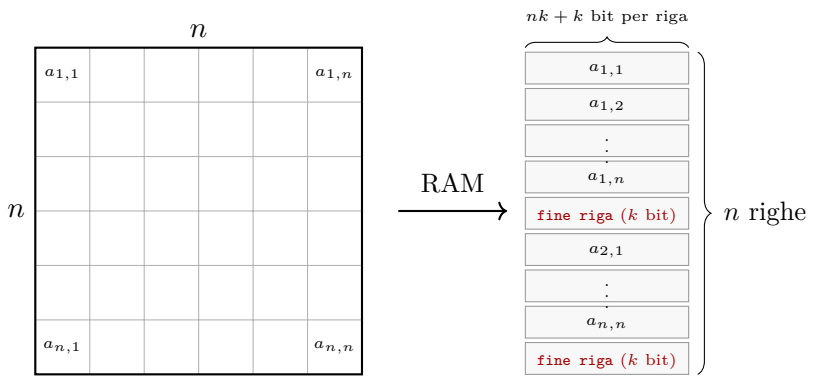
Esempi di indicatori corretti:

- **Array di n elementi:** indicatore $= n$. ✓
- **Matrice $n \times n$:** indicatore $= n$ (la dimensione reale $n \cdot k(n+1)$ è polinomiale in n). ✓
- **Intero n:** indicatore $= \lfloor \lg n \rfloor + 1$ (numero di bit). **Non** si può usare n come indicatore perché $2^{|I|} \approx n$, relazione esponenziale. ✓

Esempio 1.2 (Matrice $n \times n$). Sia A una matrice $n \times n$ di interi, dove ogni intero richiede k bit. La RAM memorizza la matrice riga per riga.

Ogni riga occupa nk bit per gli interi, più k bit per il marcatore di fine riga. L'intera matrice richiede:

$$|I| = n(n+1)k \text{ bit} = n(kn+k) \text{ bit} = \boxed{kn^2 + kn \text{ bit}} = \Theta(n)$$



Si usa come indicatore $|I| = n$. È **corretto** perché la dimensione reale $kn^2 + kn$ è un **polinomio in** n.

Nota

L'errore comune è usare il *valore* di un intero n come dimensione dell'input invece del numero di bit necessari a rappresentarlo. Questo porta a sottostimare la complessità.

Esempio: il numero 1000 ha valore 1000, ma occupa solo $\lceil \log_2(1000) \rceil = 10$ bit. La dimensione dell'input è 10, non 1000!

Analisi di InsertionSort

c_k = costo esecuzione istruzione alla riga k. $t_i - 1$ = # cicli **while** nel i-esimo ciclo **for**.

	INSERTION-SORT(A, n)	*cost*	*times*
1	**for** $i = 2$ **to** n	c_1	n
2	$key = A[i]$	c_2	$n-1$
3	`// Insert` $A[i]$ `into the sorted subarray` $A[1:i-1]$	0	$n-1$
4	$j = i - 1$	c_4	$n-1$
5	**while** $j > 0$ **and** $A[j] > key$	c_5	$\sum_{i=2}^{n} t_i$
6	$A[j+1] = A[j]$	c_6	$\sum_{i=2}^{n} (t_i - 1)$
7	$j = j - 1$	c_7	$\sum_{i=2}^{n} (t_i - 1)$
8	$A[j+1] = key$	c_8	$n-1$

Contare le iterazioni:
for $i = \ell$ **to** n $\quad\Rightarrow\quad$ $\#\text{Cicli} = n - \ell + 1$
Esempio: for $i = 2$ **to** n $\quad\Rightarrow\quad$ $n - 1$ iterazioni

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{i=2}^{n} t_i + c_6 \sum_{i=2}^{n} (t_i - 1) + c_7 \sum_{i=2}^{n} (t_i - 1) + c_8(n-1)$$

Indichiamo con c_k il costo dell'istruzione alla riga k, e con t_i il numero di volte che il ciclo `while` viene eseguito durante l'i-esimo ciclo `for`.

Nota

t_i dipende dai dati:

- Se $A[i]$ è già al posto giusto: $t_i = 1$ (un solo confronto)
- Se $A[i]$ deve andare all'inizio: $t_i = i$ (confronta con tutti)

Caso migliore: array già ordinato

Se l'array è già ordinato, il corpo del `while` non viene mai eseguito: $t_i = 1$ per ogni i. Le somme $\sum(t_i - 1) = 0$, quindi:

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\ &= \underbrace{(c_1 + c_2 + c_4 + c_5 + c_8)}_{a} n - \underbrace{(c_2 + c_4 + c_5 + c_8)}_{b} \\ &= an - b \quad \Rightarrow \quad \boxed{T(n) = \Theta(n)} \end{aligned}$$

Nota

Nel caso migliore la crescita è **lineare**: ogni elemento viene confrontato una sola volta con il suo predecessore e nessuno scambio viene effettuato.

Esempio pratico: se l'array è $[1, 2, 3, 4, 5]$, l'algoritmo fa solo $n - 1$ confronti e nessuno spostamento. Velocissimo!

Caso peggiore: array ordinato al contrario

Se l'array è in ordine decrescente, ogni $A[i]$ deve essere confrontato con **tutti** gli elementi in $A[1 : i-1]$, quindi $t_i = i$. Usando $\sum_{i=2}^{n} i = \frac{n(n+1)}{2} - 1$:

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right) + (c_6 + c_7)\frac{n(n-1)}{2} + c_8(n-1)$$

Raccogliendo i termini per grado:

$$T(n) = \underbrace{\frac{c_5 + c_6 + c_7}{2}}_{a} n^2 + \underbrace{\left(c_1 + c_2 + c_4 + \frac{c_5 - c_6 - c_7}{2} + c_8\right)}_{b} n - \underbrace{(c_2 + c_4 + c_5 + c_8)}_{c}$$

$$\Rightarrow \quad \boxed{T(n) = \Theta(n^2)}$$

Nota

Nel caso peggiore la crescita è **quadratica**: ogni nuovo elemento deve "scendere" fino in fondo alla parte già ordinata, confrontandosi con tutti gli elementi precedenti.
Esempio pratico: se l'array è $[5, 4, 3, 2, 1]$:

- Il 4 fa 1 confronto
- Il 3 fa 2 confronti
- Il 2 fa 3 confronti
- L'1 fa 4 confronti

Totale: $1 + 2 + 3 + 4 = 10 = \frac{n(n-1)}{2}$ confronti. Molto lento!

Caso	Input	Complessità
Migliore	Array già ordinato	$\Theta(n)$
Peggiore	Array ordinato inverso	$\Theta(n^2)$

Solitamente si considera come complessità di un algoritmo il tempo di esecuzione con input che rappresenta il caso peggiore.

1. È una garanzia sulla descrizione del comportamento dell'algoritmo.
2. In alcuni casi, il caso peggiore è il più frequente.
3. Il caso medio spesso è simile al caso peggiore (vedi InsertionSort).

Per InsertionSort: il caso medio è $\Theta(n^2)$ come il caso peggiore, perché in media metà degli elementi vanno spostati.

1.6 MergeSort

MergeSort è un algoritmo di ordinamento basato sul metodo **divide et impera**, più efficiente di InsertionSort nel caso peggiore: $\Theta(n \lg n)$ contro $\Theta(n^2)$.

1.6.1 Metodo Divide et Impera

Il metodo divide et impera è un paradigma di progettazione degli algoritmi che si articola in tre fasi ricorsive:

- **Divide:** il problema di dimensione n viene suddiviso in a sotto-problemi di dimensione n/b.
- **Impera:** ogni sotto-problema viene risolto ricorsivamente. La ricorsione si ferma quando il problema è sufficientemente piccolo da essere risolto direttamente (caso base).
- **Combina:** le soluzioni dei sotto-problemi vengono unite per ottenere la soluzione del problema originale.

Nota

InsertionSort usa un approccio **incrementale**: estende la soluzione di $A[1 \dots i-1]$ inserendo $A[i]$ nella posizione corretta. MergeSort usa invece **divide et impera**: risolve due metà indipendentemente e le fonde.

1.6.2 Descrizione dell'algoritmo

Applicando divide et impera al problema dell'ordinamento:

- **Divide:** l'array $A[p \dots r]$ viene spezzato in due sotto-array $A[p \dots q]$ e $A[q+1 \dots r]$, dove $q = \lfloor (p+r)/2 \rfloor$.
- **Impera:** i due sotto-array vengono ordinati ricorsivamente con MergeSort. Quando $p = r$ (sotto-array di un solo elemento), la ricorsione termina.
- **Combina:** i due sotto-array ordinati vengono fusi dall'operazione $\textsc{Merge}(A, p, q, r)$.

L'operazione chiave è $\textsc{Merge}$: dati due sotto-array adiacenti già ordinati, li fonde in un unico sotto-array ordinato.

1.6.3 Pseudocodice

Dato $A[1 \dots n]$, MergeSort si invoca con $\textsc{Merge-Sort}(A, 1, n)$.

Algorithm 8: MERGE-SORT(A, p, r)

Input: Array A, indici $p \leq r$

Output: $A[p \dots r]$ ordinato in loco

```
if p < r then
    q = ⌊(p + r)/2⌋;
    MERGE-SORT(A, p, q);
    MERGE-SORT(A, q + 1, r);
    MERGE(A, p, q, r);
```

Algorithm 9: MERGE(A, p, q, r)

Input: $A[p \dots q]$ e $A[q+1 \dots r]$ già ordinati

Output: $A[p \dots r]$ ordinato

```
n_L = q − p + 1;
n_R = r − q;
crea array L[0 ... n_L − 1] e R[0 ... n_R − 1];
for i = 0 to n_L − 1 do
    L[i] = A[p + i];
for j = 0 to n_R − 1 do
    R[j] = A[q + 1 + j];
i = 0;   j = 0;   k = p;
while i < n_L and j < n_R do
    if L[i] ≤ R[j] then
        A[k] = L[i];;
        i = i + 1;
    else
        A[k] = R[j];;
        j = j + 1;
    k = k + 1;
while i < n_L do
    A[k] = L[i];;
    i = i + 1;;
    k = k + 1;
while j < n_R do
    A[k] = R[j];;
    j = j + 1;;
    k = k + 1;
```

Nota

Questa è la versione più semplice di MERGE. Nel Cormen (ed. IV) e in rete esistono versioni più compatte che usano la tecnica della **sentinella**: si aggiunge $+\infty$ in fondo a L e R, eliminando i due cicli **while** finali.

1.6.4 Correttezza: Invariante di ciclo di Merge

Definizione 1.9 (Invariante di MERGE)**.** All'inizio di ogni iterazione del ciclo **while** principale, $A[p \dots k-1]$ contiene, ordinati, gli $i+j$ elementi minori di tutti gli elementi in $L[0 \dots n_L - 1]$ e $R[0 \dots n_R - 1]$. $L[i]$ e $R[j]$ sono gli elementi più piccoli dei rispettivi array non ancora copiati in A.

Dimostrazione.

1. **Inizializzazione:** $k = p$, quindi $A[p \dots k-1]$ è vuoto (0 elementi copiati). Con $i = j = 0$, $L[0]$ e $R[0]$ sono i minimi non ancora copiati. L'invariante è vera. ✓
2. **Conservazione:** supponiamo $L[i] \leq R[j]$ (il caso $L[i] > R[j]$ è simmetrico). $L[i]$ è il minimo tra tutti gli elementi non ancora copiati, quindi viene copiato in $A[k]$. Incrementando k e i, l'array $A[p \dots k-1]$ contiene ora $i+j$ elementi ordinati e $L[i]$ è il nuovo minimo non copiato di L. L'invariante è mantenuta. ✓
3. **Conclusione:** il ciclo termina quando $i = n_L$ oppure $j = n_R$, cioè quando uno dei due array è esaurito. I rimanenti elementi dell'altro array (già ordinati e tutti maggiori di quelli copiati) vengono aggiunti dai due cicli **while** finali. Al termine, $A[p \dots r]$ è ordinato e contiene gli stessi elementi originali. ✓

□

1.6.5 Complessità

Complessità di Merge. Sia $n = r - p + 1$ la dimensione del sotto-array da fondere:

- Copiare in L e R: $n_L + n_R = n$ operazioni
- Ciclo **while** principale: al più n iterazioni
- Cicli **while** finali: al più n iterazioni totali

$$T_{\text{MERGE}}(n) = \Theta(n)$$

Equazione di ricorrenza di MergeSort. Con $a = 2$ sotto-problemi di dimensione $n/2$, costo di divisione $D(n) = \Theta(1)$ e costo di fusione $C(n) = \Theta(n)$:

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1 \\ 2\,T(n/2) + \Theta(n) & \text{altrimenti} \end{cases}$$

Applicando il **Teorema dell'Esperto**:

$$\boxed{T(n) = \Theta(n \lg n)} \qquad \text{dove } \lg = \log_2$$

Soluzione per albero di ricorsione.

Nota

Ad ogni livello i dell'albero vi sono 2^i sotto-problemi di dimensione $n/2^i$, ciascuno con costo di fusione $\Theta(n/2^i)$. Il costo totale per livello è:

$$2^i \cdot \Theta\left(\frac{n}{2^i}\right) = \Theta(n)$$

Il numero di livelli è $\lg n + 1$ (le foglie hanno dimensione 1, con $n/2^{\lg n} = 1$). Sommando su tutti i livelli:

$$T(n) = (\lg n + 1) \cdot \Theta(n) = \Theta(n \lg n)$$

1.6.6 Confronto con InsertionSort

Proprietà	**InsertionSort**	**MergeSort**
Paradigma	Incrementale	Divide et impera
Caso peggiore	$\Theta(n^2)$	$\Theta(n \lg n)$
Caso migliore	$\Theta(n)$	$\Theta(n \lg n)$
Caso medio	$\Theta(n^2)$	$\Theta(n \lg n)$
Ordinamento in-place	Sì	No
Memoria aggiuntiva	$\Theta(1)$	$\Theta(n)$
Stabile	Sì	Sì

1.7 Tasso di crescita e ordine di grandezza

Finalmente parliamo della notazione O, Ω, Θ! Queste sono le notazioni fondamentali per descrivere l'efficienza degli algoritmi ignorando i dettagli irrilevanti.

Nell'analisi di InsertionSort si sono usate costanti c_k per rappresentare i costi astratti delle operazioni nel modello RAM. Queste costanti possono essere raggruppate: ad esempio $(c_2 + c_4 + c_5 + c_8)$ diventa una costante generica a.

Ultima astrazione: è più importante valutare la **velocità con cui cresce** il tempo di esecuzione (*tasso di crescita*) che i dettagli delle costanti.

Di un polinomio $T(n) = an^2 + bn + c$ è sufficiente considerare solo il termine dominante an^2. Visto che la costante a è astratta, la si assorbe nell'ordine di grandezza:

$$T(n) = an^2 + bn + c \approx an^2 \implies \Theta(n^2)$$

Nota

Perché ignoriamo le costanti?
Immagina due algoritmi:

- A: $T_A(n) = 1000n$
- B: $T_B(n) = n^2$

Per n piccoli, A sembra peggio. Ma per $n > 1000$, B diventa molto più lento! Il termine n^2 "domina" alla lunga, quindi diciamo che B è $\Theta(n^2)$ e A è $\Theta(n)$.

Un algoritmo è considerato **più efficiente** di un altro se il suo tempo di computazione nel caso peggiore ha un **tasso di crescita inferiore**.

$T(n)$	Ordine di grandezza	Nome
$an + b$	$\Theta(n)$	Lineare
$an^2 + bn + c$	$\Theta(n^2)$	Quadratica
$an^3 + \cdots$	$\Theta(n^3)$	Cubica
$a \lg n + b$	$\Theta(\lg n)$	Logaritmica
$an \lg n + b$	$\Theta(n \lg n)$	Linearitmica

Scala di velocità (dal più veloce al più lento):

$$\Theta(1) < \Theta(\log n) < \Theta(n) < \Theta(n \log n) < \Theta(n^2) < \Theta(n^3) < \Theta(2^n) < \Theta(n!)$$

Regola pratica: un algoritmo $\Theta(n \log n)$ è "veloce", $\Theta(n^2)$ è "accettabile per n piccoli", $\Theta(2^n)$ è "impossibile per $n > 30$".

1.7.1 MergeSort (Divide et impera)

MergeSort usa una strategia completamente diversa da InsertionSort: invece di costruire la soluzione incrementalmente, **divide** il problema in pezzi, **risolve** i pezzi ricorsivamente, poi **combina** le soluzioni. Questa è la tecnica "Divide et Impera"!

1. **Dividi:** spezza in due metà.
2. **Impera:** ordina ricorsivamente.
3. **Combina:** fondi in $\Theta(n)$.

$$T(n) = 2\,T(n/2) + \Theta(n) \;\Rightarrow\; T(n) = \Theta(n \log n)$$

Nota

Come funziona MergeSort?
Esempio: ordina $[38, 27, 43, 3]$

1. **Dividi**: $[38, 27]$ e $[43, 3]$
2. **Dividi ancora**: $[38]$, $[27]$, $[43]$, $[3]$
3. **Combina**: $[27, 38]$ e $[3, 43]$
4. **Combina finale**: $[3, 27, 38, 43]$

La **fusione** (merge) di due array ordinati in un array ordinato si fa in tempo lineare confrontando gli elementi più piccoli di ciascun array.

Algoritmo	Caso peggiore	Strategia
InsertionSort	$\Theta(n^2)$	Incrementale
MergeSort	$\Theta(n \log n)$	Divide et impera

Differenza drammatica:
Per $n = 1.000.000$ elementi:

- InsertionSort: $\approx 10^{12}$ operazioni $\rightarrow$ ore
- MergeSort: $\approx 2 \times 10^7$ operazioni $\rightarrow$ secondi

MergeSort è circa **50.000 volte più veloce**!

1.7.2 Il problema dell'assunzione

1.7.3 Variabili casuali indicatrici

1.7.4 Algoritmi randomizzati

1.8 Calcolo della complessità

Questa sezione riassume le tecniche matematiche per calcolare la complessità. Studia bene il Master Theorem: è fondamentale per analizzare algoritmi ricorsivi!

1.8.1 Notazione asintotica

Definizione 1.10 (O, Ω, Θ)**.** Date $f, g : \mathbb{N} \to \mathbb{R}^+$:

- $f = O(g)$: $\exists\, c, n_0$ t.c. $f(n) \leq c\, g(n)$ per $n \geq n_0$ (*limite superiore*).
- $f = \Omega(g)$: $\exists\, c, n_0$ t.c. $f(n) \geq c\, g(n)$ per $n \geq n_0$ (*limite inferiore*).
- $f = \Theta(g)$: $f = O(g)$ e $f = \Omega(g)$ (*limite stretto*).

In parole semplici:

- $f = O(g)$ significa "f cresce **al massimo** come g" (limite superiore)
- $f = \Omega(g)$ significa "f cresce **almeno** come g" (limite inferiore)
- $f = \Theta(g)$ significa "f cresce **esattamente** come g" (limite stretto)

Analogia: se $g(n) = n^2$:

- $10n^2 + 5n = O(n^2)$ ✓ (cresce come n^2)
- $10n^2 + 5n = O(n^3)$ ✓ (cresce meno di n^3)
- $10n^2 + 5n = \Theta(n^2)$ ✓ (cresce esattamente come n^2)
- $10n^2 + 5n = \Theta(n)$ × (cresce più di n)

Ordine crescente:

$$O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(2^n) \subset O(n!)$$

1.8.2 Regole per algoritmi iterativi

- **Sequenza:** $O(\max(f, g))$.
- **Ciclo semplice** di n iterazioni a corpo $O(1)$: $O(n)$.
- **Cicli annidati:** prodotto dei range. Due cicli $1 \ldots n$: $O(n^2)$.
- **Passo moltiplicativo** (i raddoppia): $O(\log n)$.
- **If-else:** massimo dei due rami.

Regole pratiche per analizzare cicli:

1. **Un ciclo** che va da 1 a n: $O(n)$
2. **Due cicli annidati** 1 a n: $O(n) \times O(n) = O(n^2)$
3. **Ciclo che dimezza** (i raddoppia o n si dimezza): $O(\log n)$
4. **If-else**: prendi il ramo più costoso

Esempio (limite variabile):

```
for (int i = 1; i <= n; i++)
for (int j = 1; j <= i; j++)
op(); // O(1)
```

Costo: $\sum_{i=1}^{n} i = \dfrac{n(n+1)}{2} = \Theta(n^2)$.

Nota

Il ciclo interno fa 1 operazione quando $i = 1$, 2 operazioni quando $i = 2$, ..., n operazioni quando $i = n$. Totale: $1 + 2 + \cdots + n = n(n+1)/2 = \Theta(n^2)$.

1.8.3 Ricorrenze: forma generale

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n), \quad T(1) = \Theta(1)$$

a = numero sotto-chiamate, b = fattore riduzione, $f(n)$ = costo divisione + combinazione.

Nota

Significato dei parametri:

- a = quante volte chiamiamo ricorsivamente la funzione
- b = di quanto riduciamo la dimensione del problema
- $f(n)$ = costo per dividere e ricombinare

Esempio MergeSort: $T(n) = 2T(n/2) + \Theta(n)$

- $a = 2$ (ordiniamo due metà)
- $b = 2$ (dividiamo in due)
- $f(n) = \Theta(n)$ (costo della fusione)

1.8.4 Metodo della sostituzione

1. **Ipotizzare** la forma, es. $T(n) = O(n \log n)$.

2. **Sostituire** nella ricorrenza assumendo l'ipotesi vera per $T(n/b)$.
3. **Verificare** la disuguaglianza scegliendo le costanti.

Esempio: $T(n) = 2T(n/2) + n$, ipotesi $T(n) \leq cn \log n$:

$$T(n) \leq 2 \cdot \frac{cn}{2} \log \frac{n}{2} + n = cn \log n - cn + n \leq cn \log n \quad (c \geq 1).$$

Il metodo della sostituzione richiede di "indovinare" la soluzione e poi verificarla matematicamente. Con la pratica imparerai a riconoscere i pattern comuni.

1.8.5 Metodo dell'albero di ricorsione

1. Ogni nodo al livello k ha costo $f(n/b^k)$ e genera a figli.
2. Numero livelli: $\log_b n$. Foglie: $n^{\log_b a}$.
3. Sommare i costi per livello.

Esempio: $T(n) = 3T(n/4) + cn^2$. Costo per livello k: $c\,n^2(3/16)^k$.

$$T(n) < cn^2 \sum_{k=0}^{\infty} \left(\frac{3}{16}\right)^k = cn^2 \cdot \frac{16}{13} = O(n^2).$$

Nota

L'albero di ricorsione visualizza come si sviluppano le chiamate ricorsive. Ogni livello contribuisce un costo, e sommando tutti i livelli otteniamo il costo totale.

1.8.6 Master Theorem

Teorema 1.2 (Master Theorem)**.** *Data $T(n) = a\,T(n/b) + f(n)$, posto $p = \log_b a$:*

1. $f(n) = O(n^{p-\varepsilon}) \Rightarrow T(n) = \Theta(n^p)$ (foglie dominano)
2. $f(n) = \Theta(n^p \log^k n)$, $k \geq 0 \Rightarrow T(n) = \Theta(n^p \log^{k+1} n)$ (pareggio)
3. $f(n) = \Omega(n^{p+\varepsilon})$ *e* $a\,f(n/b) \leq c\,f(n)$, $c < 1 \Rightarrow T(n) = \Theta(f(n))$ (radice domina)

Come usare il Master Theorem:
Passo 1: Calcola $p = \log_b a$ (quante "foglie" ci sono)
Passo 2: Confronta $f(n)$ con n^p:

- Se $f(n)$ è **polinomialmente più piccolo** di n^p $\to$ Caso 1
- Se $f(n) \approx n^p$ (a meno di log) $\to$ Caso 2
- Se $f(n)$ è **polinomialmente più grande** di n^p $\to$ Caso 3

Esempi applicativi:

Ricorrenza	$p = \log_b a$	Caso	Soluzione
$2T(n/2) + \Theta(n)$	1	2	$\Theta(n \log n)$
$2T(n/2) + \Theta(n^2)$	1	3	$\Theta(n^2)$
$4T(n/2) + \Theta(n)$	2	1	$\Theta(n^2)$
$2T(n/3) + \Theta(1)$	$\log_3 2$	1	$\Theta(n^{\log_3 2})$
$T(n/2) + \Theta(1)$	0	2	$\Theta(\log n)$

Esempi commentati:
MergeSort: $T(n) = 2T(n/2) + \Theta(n)$

- $p = \log_2 2 = 1$
- $f(n) = n = \Theta(n^1) \to$ Caso 2
- Soluzione: $\Theta(n \log n)$

Ricerca binaria: $T(n) = T(n/2) + \Theta(1)$

- $p = \log_2 1 = 0$
- $f(n) = 1 = \Theta(n^0) \to$ Caso 2
- Soluzione: $\Theta(\log n)$

Osservazione 2. Il Master Theorem **non si applica** se $f(n)$ non è polinomialmente confrontabile con n^p o se la forma non è $a\,T(n/b) + f(n)$.

1.8.7 Ricorrenze non standard (telescoping)

Ricorrenza	Soluzione
$T(n) = T(n-1) + O(1)$	$\Theta(n)$
$T(n) = T(n-1) + O(n^c)$	$\Theta(n^{c+1})$
$T(n) = 2T(n-1) + O(1)$	$\Theta(2^n)$
$T(n) \leq T(\alpha n) + T((1-\alpha)n) + O(n)$	$O(n \log n)$

Telescoping: $T(n) = T(n-1) + f(n) = T(1) + \sum_{k=2}^{n} f(k)$. Per $f(k) = k^c$: somma $= \Theta(n^{c+1})$.

Nota

Il **telescoping** è utile quando la ricorrenza riduce di 1 invece che dividere per 2. Si "apre" la ricorsione scrivendo tutti i termini e poi si somma.

2. Può tornare utile

Questa sezione contiene materiale di riferimento (appendici) e domande d'esame.

2.1 Appendice A (Sommatorie)

2.1.1 Formule fondamentali

$$\sum_{i=1}^{n} 1 = n$$

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2)$$

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6} = \Theta(n^3)$$

$$\sum_{i=1}^{n} i^k = \Theta(n^{k+1}) \quad \text{per } k \geq 0$$

$$\sum_{i=0}^{n} x^i = \frac{x^{n+1}-1}{x-1} \quad \text{per } x \neq 1$$

$$\sum_{i=0}^{\infty} x^i = \frac{1}{1-x} \quad \text{per } |x| < 1$$

2.1.2 Serie geometrica

Nota

La serie geometrica è fondamentale nell'analisi degli algoritmi ricorsivi.

Formula generale:

$$\sum_{i=0}^{n} ar^i = a \cdot \frac{r^{n+1} - 1}{r - 1}$$

Caso particolare importante ($r = 2$):

$$\sum_{i=0}^{n} 2^i = 2^{n+1} - 1$$

Serie infinita convergente ($|r| < 1$):

$$\sum_{i=0}^{\infty} ar^i = \frac{a}{1 - r}$$

2.1.3 Serie armonica

$$H_n = \sum_{i=1}^{n} \frac{1}{i} = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} = \Theta(\log n)$$

Nota

La serie armonica cresce **logaritmicamente**. Utile per analizzare:

- Quicksort (numero medio di confronti)
- Hashing (lunghezza catene)
- Alberi bilanciati (altezza media)

Approssimazione: $H_n \approx \ln n + \gamma$ dove $\gamma \approx 0.5772$ (costante di Eulero-Mascheroni)

2.1.4 Tecniche di manipolazione

1. Separazione dei termini:

$$\sum_{i=1}^{n} (a_i + b_i) = \sum_{i=1}^{n} a_i + \sum_{i=1}^{n} b_i$$

2. Estrazione di costanti:

$$\sum_{i=1}^{n} c \cdot a_i = c \sum_{i=1}^{n} a_i$$

3. Telescoping (sommatoria telescopica):

$$\sum_{i=1}^{n}(a_i - a_{i-1}) = a_n - a_0$$

Esempio 2.1 (Telescoping applicato).

$$\sum_{i=1}^{n}\frac{1}{i(i+1)} = \sum_{i=1}^{n}\left(\frac{1}{i} - \frac{1}{i+1}\right) = 1 - \frac{1}{n+1} = \frac{n}{n+1}$$

4. Cambio di variabile:

Se $j = n - i$, allora:

$$\sum_{i=0}^{n} f(i) = \sum_{j=n}^{0} f(n-j) = \sum_{j=0}^{n} f(n-j)$$

2.1.5 Approssimazione con integrali

Per f crescente:

$$\int_1^n f(x)\,dx \leq \sum_{i=1}^{n} f(i) \leq f(n) + \int_1^n f(x)\,dx$$

Esempio 2.2 (Somma di potenze).

$$\sum_{i=1}^{n} i^k \approx \int_1^n x^k\,dx = \frac{n^{k+1}}{k+1}$$
$$\Rightarrow \sum_{i=1}^{n} i^k = \Theta(n^{k+1})$$

2.2 Appendice B (Insiemi, relazioni, funzioni, grafi, alberi)

2.2.1 Insiemi

Definizione 2.1 (Insieme)**.** Collezione non ordinata di elementi distinti. Notazione: $A = \{1, 2, 3\}$

Operazioni fondamentali:

- **Unione:** $A \cup B = \{x : x \in A \text{ oppure } x \in B\}$
- **Intersezione:** $A \cap B = \{x : x \in A \text{ e } x \in B\}$
- **Differenza:** $A \setminus B = \{x : x \in A \text{ e } x \notin B\}$
- **Complemento:** $\overline{A} = \{x : x \notin A\}$ (rispetto a un universo)

Nota

Cardinalità: $|A|$ = numero di elementi in A
Insieme delle parti: $\mathcal{P}(A)$ = insieme di tutti i sottoinsiemi di A
Se $|A| = n$ allora $|\mathcal{P}(A)| = 2^n$

2.2.2 Relazioni

Definizione 2.2 (Relazione)**.** Una relazione R da A a B è un sottoinsieme di $A \times B$ (prodotto cartesiano).

Scriviamo aRb se $(a, b) \in R$.

Proprietà di relazioni su A (relazione binaria):

- **Riflessiva:** $\forall a \in A : aRa$
- **Simmetrica:** $\forall a, b \in A : aRb \Rightarrow bRa$
- **Transitiva:** $\forall a, b, c \in A : (aRb \wedge bRc) \Rightarrow aRc$
- **Antisimmetrica:** $\forall a, b \in A : (aRb \wedge bRa) \Rightarrow a = b$

Definizione 2.3 (Relazione di equivalenza)**.** Una relazione è di **equivalenza** se è riflessiva, simmetrica e transitiva.

Esempio: uguaglianza (=), congruenza modulo n

Definizione 2.4 (Ordine parziale)**.** Una relazione è di **ordine parziale** se è riflessiva, antisimmetrica e transitiva.

Esempio: $\leq$ sui numeri reali, $\subseteq$ sugli insiemi

2.2.3 Funzioni

Definizione 2.5 (Funzione)**.** Una funzione $f : A \to B$ assegna a ogni elemento di A esattamente un elemento di B.

- A = **dominio**
- B = **codominio**
- $f(A) = \{f(a) : a \in A\}$ = **immagine**

Tipi di funzioni:

- **Iniettiva** (uno-a-uno): $f(a_1) = f(a_2) \Rightarrow a_1 = a_2$
- **Suriettiva** (sopra): $\forall b \in B, \exists a \in A : f(a) = b$

- **Biettiva** (biunivoca): iniettiva e suriettiva

Nota

Una funzione biettiva ha **funzione inversa** $f^{-1} : B \to A$ tale che:

$$f^{-1}(f(a)) = a \quad \text{e} \quad f(f^{-1}(b)) = b$$

2.2.4 Grafi

Definizione 2.6 (Grafo)**.** Un grafo $G = (V, E)$ consiste di:

- V = insieme di **vertici** (o nodi)
- $E \subseteq V \times V$ = insieme di **archi**

Tipi di grafi:

- **Grafo non orientato:** $(u, v) \in E \Leftrightarrow (v, u) \in E$
- **Grafo orientato** (digrafo): gli archi hanno una direzione
- **Grafo pesato:** ogni arco ha un peso $w : E \to \mathbb{R}$

Terminologia:

- **Grado** di v: numero di archi incidenti
  - Grafo orientato: *grado entrante* + *grado uscente*
- **Cammino:** sequenza di vertici $v_1, v_2, \ldots, v_k$ con $(v_i, v_{i+1}) \in E$
- **Ciclo:** cammino con $v_1 = v_k$
- **Connesso:** esiste un cammino tra ogni coppia di vertici

Rappresentazioni:

1. **Matrice di adiacenza** $(n \times n)$:

$$A[i][j] = \begin{cases} 1 & \text{se } (i, j) \in E \\ 0 & \text{altrimenti} \end{cases}$$

   Spazio: $\Theta(n^2)$, accesso arco: $O(1)$

2. **Liste di adiacenza:** Array di n liste, $\text{Adj}[u]$ contiene i vicini di u

   Spazio: $\Theta(n + m)$, iterare vicini: $O(\deg(u))$

2.2.5 Alberi

Definizione 2.7 (Albero)**.** Un albero è un grafo **connesso** e **aciclico**.

Equivalentemente:

- Grafo connesso con n nodi e $n-1$ archi
- Esiste un unico cammino tra ogni coppia di nodi

Albero radicato: albero con un nodo speciale detto **radice**.

- **Padre/Figlio:** u è padre di v se $(u, v) \in E$ e u è più vicino alla radice
- **Foglia:** nodo senza figli
- **Altezza:** lunghezza del cammino più lungo dalla radice a una foglia
- **Profondità** di v: distanza dalla radice a v

Definizione 2.8 (Albero binario)**.** Albero radicato in cui ogni nodo ha al più 2 figli (sinistro e destro).

Albero binario completo: tutte le foglie allo stesso livello, ogni nodo interno ha 2 figli.

Un albero binario completo di altezza h ha $2^{h+1} - 1$ nodi.

Visite di alberi binari:

- **Preorder:** radice $\rightarrow$ sinistro $\rightarrow$ destro
- **Inorder:** sinistro $\rightarrow$ radice $\rightarrow$ destro
- **Postorder:** sinistro $\rightarrow$ destro $\rightarrow$ radice

2.3 Appendice C (Calcolo combinatorio e probabilità)

2.3.1 Principi di conteggio

1. Principio del prodotto:

Se un evento può avvenire in m modi e un altro in n modi, allora entrambi possono avvenire in $m \times n$ modi.

Esempio 2.3. Quante stringhe binarie di lunghezza 3?

Ogni posizione: 2 scelte $\Rightarrow$ totale: $2 \times 2 \times 2 = 8$

2. Principio della somma:

Se un evento può avvenire in m modi disgiunti o in n modi disgiunti, può avvenire in $m + n$ modi.

2.3.2 Permutazioni e combinazioni

Definizione 2.9 (Permutazione)**.** Numero di modi per ordinare n oggetti distinti:

$$P(n) = n! = n \times (n-1) \times \cdots \times 2 \times 1$$

Definizione 2.10 (Permutazione con ripetizioni)**.** Se tra n oggetti ci sono gruppi di $n_1, n_2, \ldots, n_k$ oggetti identici:

$$\frac{n!}{n_1! \cdot n_2! \cdot \ldots \cdot n_k!}$$

Esempio 2.4. Permutazioni della parola "MISSISSIPPI" (11 lettere: 4 I, 4 S, 2 P, 1 M):

$$\frac{11!}{4! \cdot 4! \cdot 2! \cdot 1!} = 34650$$

Definizione 2.11 (Disposizioni)**.** Numero di modi per scegliere k oggetti da n tenendo conto dell'ordine:

$$D(n,k) = \frac{n!}{(n-k)!} = n \times (n-1) \times \cdots \times (n-k+1)$$

Definizione 2.12 (Combinazioni (coefficiente binomiale))**.** Numero di modi per scegliere k oggetti da n **senza** considerare l'ordine:

$$C(n,k) = \binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Proprietà dei coefficienti binomiali:

$$\begin{aligned}
\binom{n}{0} &= \binom{n}{n} = 1 \\
\binom{n}{k} &= \binom{n}{n-k} \quad \text{(simmetria)} \\
\binom{n}{k} &= \binom{n-1}{k-1} + \binom{n-1}{k} \quad \text{(ricorsione - Pascal)} \\
\sum_{k=0}^{n} \binom{n}{k} &= 2^n
\end{aligned}$$

2.3.3 Probabilità

Definizione 2.13 (Spazio campionario)**.** Insieme Ω di tutti i possibili risultati di un esperimento.

Definizione 2.14 (Evento)**.** Sottoinsieme $A \subseteq \Omega$ dello spazio campionario.

Definizione 2.15 (Probabilità)**.** Funzione $P : \mathcal{P}(\Omega) \to [0,1]$ tale che:

- $P(\Omega) = 1$
- $P(A \cup B) = P(A) + P(B)$ se $A \cap B = \emptyset$ (additività)

Caso equiprobabile:

$$P(A) = \frac{|A|}{|\Omega|} = \frac{\text{casi favorevoli}}{\text{casi possibili}}$$

Regole fondamentali:

- $P(\emptyset) = 0$
- $P(\overline{A}) = 1 - P(A)$ (complemento)
- $P(A \cup B) = P(A) + P(B) - P(A \cap B)$ (unione)

2.3.4 Probabilità condizionata e indipendenza

Definizione 2.16 (Probabilità condizionata)**.** La probabilità di A dato B (con $P(B) > 0$):

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

Teorema 2.1 (Regola del prodotto)**.**

$$P(A \cap B) = P(A \mid B) \cdot P(B) = P(B \mid A) \cdot P(A)$$

Definizione 2.17 (Eventi indipendenti)**.** A e B sono indipendenti se:

$$P(A \cap B) = P(A) \cdot P(B)$$

Equivalentemente: $P(A \mid B) = P(A)$

Teorema 2.2 (Teorema di Bayes)**.**

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$

2.3.5 Variabili casuali e valore atteso

Definizione 2.18 (Variabile casuale)**.** Funzione $X : \Omega \to \mathbb{R}$ che assegna un valore numerico a ogni risultato.

Definizione 2.19 (Valore atteso)**.** Media pesata dei valori:

$$E[X] = \sum_x x \cdot P(X = x)$$

Proprietà del valore atteso:

- $E[aX + b] = aE[X] + b$ (linearità)
- $E[X + Y] = E[X] + E[Y]$ (sempre vera)

- $E[XY] = E[X] \cdot E[Y]$ (solo se X e Y indipendenti)

Definizione 2.20 (Variabile indicatrice)**.** Variabile casuale che vale 1 se un evento si verifica, 0 altrimenti.

$$I_A = \begin{cases} 1 & \text{se } A \text{ si verifica} \\ 0 & \text{altrimenti} \end{cases}$$

Proprietà fondamentale: $E[I_A] = P(A)$

Esempio 2.5 (Numero atteso di confronti in QuickSort)*.* Usando variabili indicatrici si dimostra che il numero atteso di confronti è $\Theta(n \log n)$.

2.4 Appendice D (Matrici)

2.4.1 Definizioni base

Definizione 2.21 (Matrice)**.** Una matrice A di dimensione $m \times n$ è una tabella di m righe e n colonne:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

Notazione: $A = (a_{ij})$ dove a_{ij} è l'elemento in posizione (i, j)

Tipi speciali:

- **Matrice quadrata:** $m = n$
- **Matrice identità** I_n: diagonale di 1, resto di 0

$$I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- **Matrice diagonale:** solo la diagonale può essere non-zero
- **Matrice triangolare superiore:** $a_{ij} = 0$ per $i > j$
- **Matrice simmetrica:** $A = A^T$ (vale $a_{ij} = a_{ji}$)

2.4.2 Operazioni fondamentali

1. Somma: (solo se A e B hanno stesse dimensioni)

$$(A + B)_{ij} = a_{ij} + b_{ij}$$

Complessità: $\Theta(mn)$

2. Prodotto per scalare:

$$(cA)_{ij} = c \cdot a_{ij}$$

Complessità: $\Theta(mn)$

3. Trasposizione: scambia righe con colonne

$$(A^T)_{ij} = a_{ji}$$

Se A è $m \times n$, allora A^T è $n \times m$

Complessità: $\Theta(mn)$

4. Prodotto: A è $m \times n$, B è $n \times p$

$$(AB)_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$$

AB è $m \times p$

Complessità (algoritmo naive): $\Theta(mnp)$

Per matrici quadrate $n \times n$: $\Theta(n^3)$

2.4.3 Proprietà del prodotto

- **Associativa:** $(AB)C = A(BC)$
- **Distributiva:** $A(B + C) = AB + AC$
- **NON commutativa:** in generale $AB \neq BA$
- $(AB)^T = B^T A^T$ (inversione ordine)
- $IA = AI = A$ (elemento neutro)

2.4.4 Determinante e matrice inversa

Definizione 2.22 (Determinante)**.** Per matrice 2×2:

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc$$

Per matrici $n \times n$: sviluppo di Laplace (ricorsivo)

Proprietà:

- $\det(AB) = \det(A) \cdot \det(B)$
- $\det(A^T) = \det(A)$

- $\det(I) = 1$
- $\det(cA) = c^n \det(A)$ per matrice $n \times n$

Definizione 2.23 (Matrice inversa)**.** A è invertibile se esiste A^{-1} tale che:

$$AA^{-1} = A^{-1}A = I$$

Condizione necessaria e sufficiente: $\det(A) \neq 0$

Proprietà della matrice inversa:

- $(A^{-1})^{-1} = A$
- $(AB)^{-1} = B^{-1}A^{-1}$ (inversione ordine)
- $(A^T)^{-1} = (A^{-1})^T$

2.4.5 Applicazioni agli algoritmi

1. Grafi: matrice di adiacenza

$A^k[i][j]$ = numero di cammini di lunghezza k da i a j

2. Programmazione dinamica: Floyd-Warshall usa prodotto matriciale generalizzato

3. Complessità prodotto matriciale:

- Algoritmo standard: $\Theta(n^3)$
- Algoritmo di Strassen: $\Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$
- Algoritmi avanzati: $O(n^{2.37})$ (teorici, non pratici)

2.5 Domande d'esame (Appello 2)

Queste sono domande reali dell'esame.[3]

Definizione di algoritmo

D: Definizione sufficiente di algoritmo?
R: Procedura ben definita che produce output da input in tempo finito.

Problema computazionale

D: Cos'è un problema computazionale?
R: Relazione input-output desiderata. Il problema dice *cosa*, l'algoritmo dice *come*.

[3] Se si studiano solo queste l'esame si passa solo se si è colpiti da un fulmine mentre si fa un salto mortale carpiato suonando il flauto

Efficienza

D: Cosa si intende per efficienza?
R: Come crescono le risorse al crescere di n (non il tempo in secondi, dipendente dall'hardware).

InsertionSort vs MergeSort

D: InsSort $= 10n^2$, MergeSort $= 32n \log_2 n$. Quando è più veloce InsSort?
R: Per n piccoli: $10n^2 < 32n \log_2 n$ solo inizialmente.

$100n^2$ vs 2^n

D: Minimo n tale che $100n^2 < 2^n$?
R: $n \geq 15$.

Invariante di ciclo

D: Cos'è un'invariante di ciclo?
R: Proprietà vera all'inizio di ogni iterazione. Si dimostra per inizializzazione, conservazione e conclusione.

Vale $\max(f, g) = \Theta(f + g)$?

R: Sì: $\max(f, g) \leq f + g \leq 2 \max(f, g)$, con $c_1 = 1$, $c_2 = 2$.

2^{n+1} e 2^{2n} vs $O(2^n)$

R: $2^{n+1} = O(2^n)$ (costante 2). $2^{2n} = 4^n \neq O(2^n)$ (richiederebbe $2^n \leq c$, impossibile).

Costo costante in RAM

R: Quando la parola di memoria ha dimensione fissa (es. 64 bit).

Prestazioni su istanze note

R: Precomputare le soluzioni e restituirle direttamente quando quell'istanza viene ricevuta.

$T(n) = T(n-1) + n^c$

R: $T(n) = \Theta(n^{c+1})$. Dim.: $\sum_{k=1}^{n} k^c \approx \int_1^n x^c \, dx = \frac{n^{c+1}}{c+1}$.

$T(n) = 2T(n/3) + 1$

R: Master Theorem, caso 1: $T(n) = \Theta(n^{\log_3 2})$ ($\log_3 2 \approx 0.63$).

Divide et impera: classe di problemi

R: Problemi divisibili in 2+ sotto-istanze con combinazione a costo al più polinomiale.

Analisi probabilistica

R: Uso del calcolo della probabilità nell'analisi degli algoritmi (non solo caso medio).

permute-without-identity

R: No: con $n = 3$ produce solo 3 permutazioni invece delle 5 attese.

Hire-Assistant: probabilità

R: Esattamente 1 assunzione: $1/n$. Esattamente n assunzioni: $1/n!$.

Generatore casuale

R: Valore intero in un intervallo con probabilità uniforme.

Algoritmo randomizzato

R: Il comportamento dipende dall'input e da valori casuali.

Ricorrenza con frazioni

D: $T(n) \leq T(10n/11) + T(n/11) + O(n)$?
R: $T(n) = O(n \log n)$. Ogni livello costa $O(n)$, profondità $O(\log n)$.

Partition su array

D: Partition su $[13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11]$, pivot $A[11]$?
R: $[9, 5, 8, 7, 4, 2, 6, \mathbf{11}, 21, 13, 19, 12]$, pivot in pos. 7.

Quicksort con elementi uguali

R: $O(n^2)$: ogni Partition genera partizione $(n-1)$ vs 0.

Partition elementi uguali: fix

R: Restituisce indice massimo. Fix: flag booleano; se tutti uguali, restituire $\lfloor (p + r)/2 \rfloor$.

Quicksort non crescente

R: Invertire il confronto in Partition: da $\leq$ a $\geq$.

Rand-Select: caso peggiore

R: Il pivot è sempre il massimo: dimensione scende di 1 a ogni passo, $O(n^2)$.

IV statistica d'ordine

D: IV statistica di $[8, 4, 8, 3, 4, 2, 3, 2]$?
R: Ordinato: $[2, 2, 3, 3, 4, 4, 8, 8]$. IV elemento $= 3$.

Eliminare duplicati

D: Eliminare duplicati in $\Theta(n \log n)$?
R: Ordinare, poi scorrere linearmente rimuovendo consecutivi uguali.

Build-Max-Heap: indice decrescente

R: Le foglie sono già heap. MaxHeapify richiede figli sistemati: si parte dal basso.

Visita inorder con pila

R: Visita in ordine (inorder), $O(n)$. Pattern: push-sinistra, pop-visita-destra.

LCS senza tabella b

```
PRINT-LCS(c, X, Y, i, j)
if c[i,j] == 0 then return
if X[i] == Y[j]
PRINT-LCS(c, X, Y, i-1, j-1)
print X[i]
else if c[i-1,j] >= c[i,j-1]
PRINT-LCS(c, X, Y, i-1, j)
else
PRINT-LCS(c, X, Y, i, j-1)
```

Costo $O(m+n)$: ogni chiamata riduce i o j di 1.

Cammino minimo vs TSP

R: Cammino minimo: sorgente $\rightarrow$ destinazione, polinomiale (Dijkstra). TSP: circuito visitando ogni nodo una volta, NP-difficile.

Prima e ultima cifra (Java)

```java
int last  = n % 10;
int first = n / (int) Math.pow(10, (int)(Math.log10(n)));
```

`T extends Comparable`

R: La ricerca binaria usa confronti: serve `compareTo()`, disponibile solo se T estende `Comparable`.

Ereditarietà e attributo omonimo

R: `super.getIdA()` per `A.idA`, `this.getIdA()` per `B.idA`. Le variabili private non sono ereditate.

Errore in makeCombination

R: `values = numbers` copia solo il riferimento locale. Fix: copia elemento per elemento.

Eccezione in try-with-resources

R: L'eccezione di `close()` è soppressa (JLS 14.20.3): diventa *suppressed exception* della prima.

Downcasting Java

D: `A value = new B(); B value1 = (A) value; value = null;`?
R: `value` → null; `value1` punta all'oggetto `B` ancora in memoria.

Stack con due code

Push i-esimo: enqueue in ausiliaria, svuota corrente nell'ausiliaria, scambia. Costo: $O(i)$. Totale n push: $O(n^2)$.
Pop: dequeue. Costo: $O(1)$.

Ricerca in matrice destrorsa

R: Ricerca binaria su ogni riga. Complessità: $O(n \log n)$.

```java
        public static <T extends Comparable<T>>
        int[] search(T[][] M, T x) {
            int n = M.length;
            for (int i = 0; i < n; i++) {
                int lo = 0, hi = n - 1;
                while (lo <= hi) {
                    int mid = (lo + hi) / 2;
                    int cmp = M[i][mid].compareTo(x);
                    if      (cmp == 0) return new int[]{i, mid};
                    else if (cmp <  0) lo = mid + 1;
                    else               hi = mid - 1;
                }
            }
            return null;
        }
```